

Assignment 3: Distributed Infrastructure, Model Serving, and CI/CD

Task 1. Single Server Deployment with CloudInit

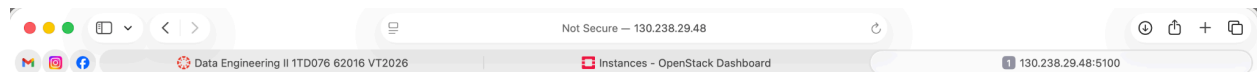
Question 1 : The application uses a three-tier architecture deployed on a single Ubuntu 22.04 VM in the Swedish Science Cloud. A Flask web server runs on port 5100 and exposes two endpoints: /accuracy and /predictions. When a user submits a POST request, Flask dispatches an asynchronous task to a Celery worker using the .delay() method. RabbitMQ serves as the message broker and queues tasks via the AMQP protocol at pyamqp://guest@localhost/. The Celery worker (workerA.py) processes each task independently. It loads a pre-trained Keras model from model.json and model.h5, runs inference on the Pima Indians Diabetes dataset, and returns results via the RabbitMQ rpc:// backend. Flask collects the result with .get() and renders it to the client. This decoupled design keeps the web server non-blocking while compute-intensive ML inference runs asynchronously in the background. The setup achieves 71.43% model accuracy.

Question 2 : The CloudInit-based contextualization strategy works but has several drawbacks. First, deployment is slow. Installing packages, cloning the repository, and setting up Python dependencies on each new VM boot takes 10 to 15 minutes before the application is available. Second, there is no guarantee of idempotency. If any runcmd step fails silently, such as from a transient network error during pip3 install, the VM boots in a broken state with no automatic recovery. Third, reproducibility is incomplete. Python packages are version-pinned (e.g., tensorflow==2.10.0, flask==2.3.1), but OS-level packages installed via apt are not locked. This allows the environment to drift across deployments. Fourth, the approach does not scale well. Each new instance must reinstall the full stack from scratch, with no layer caching or shared base image. This makes scaling slow and resource-inefficient. Fifth, sensitive configurations, like RabbitMQ credentials, are hard-coded in the source files. This is a security concern for any production deployment.

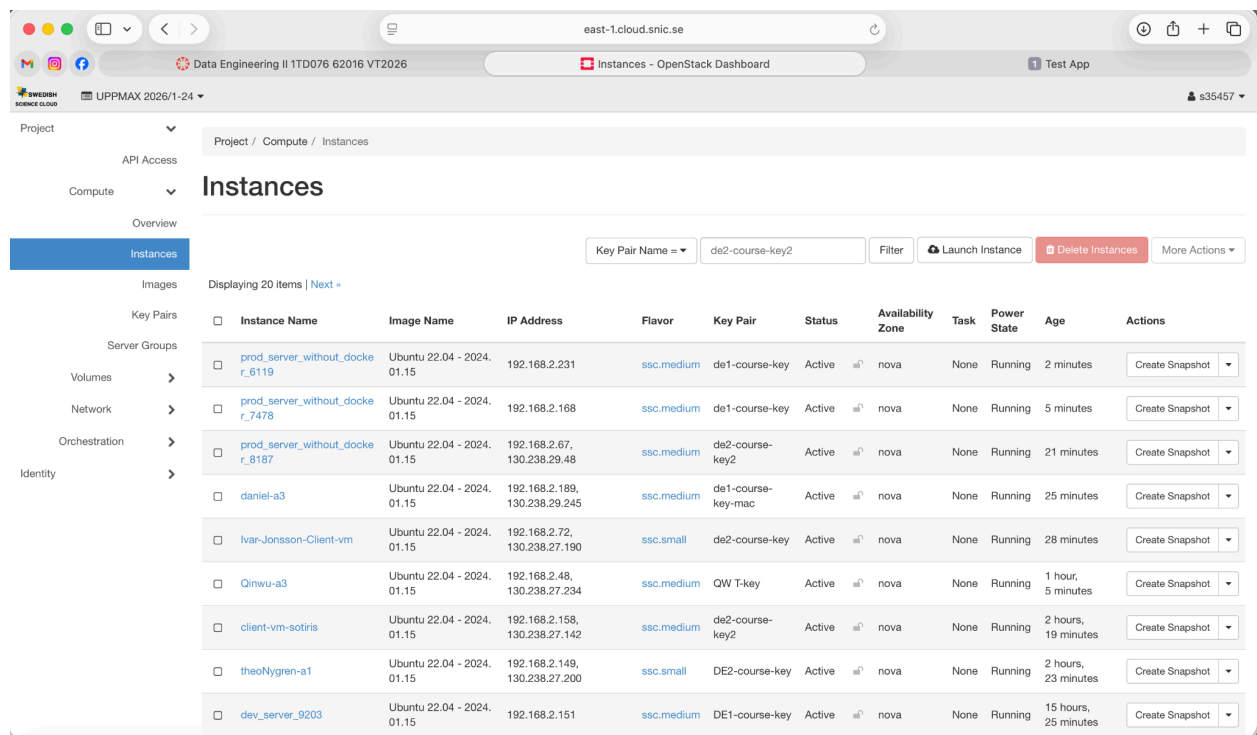
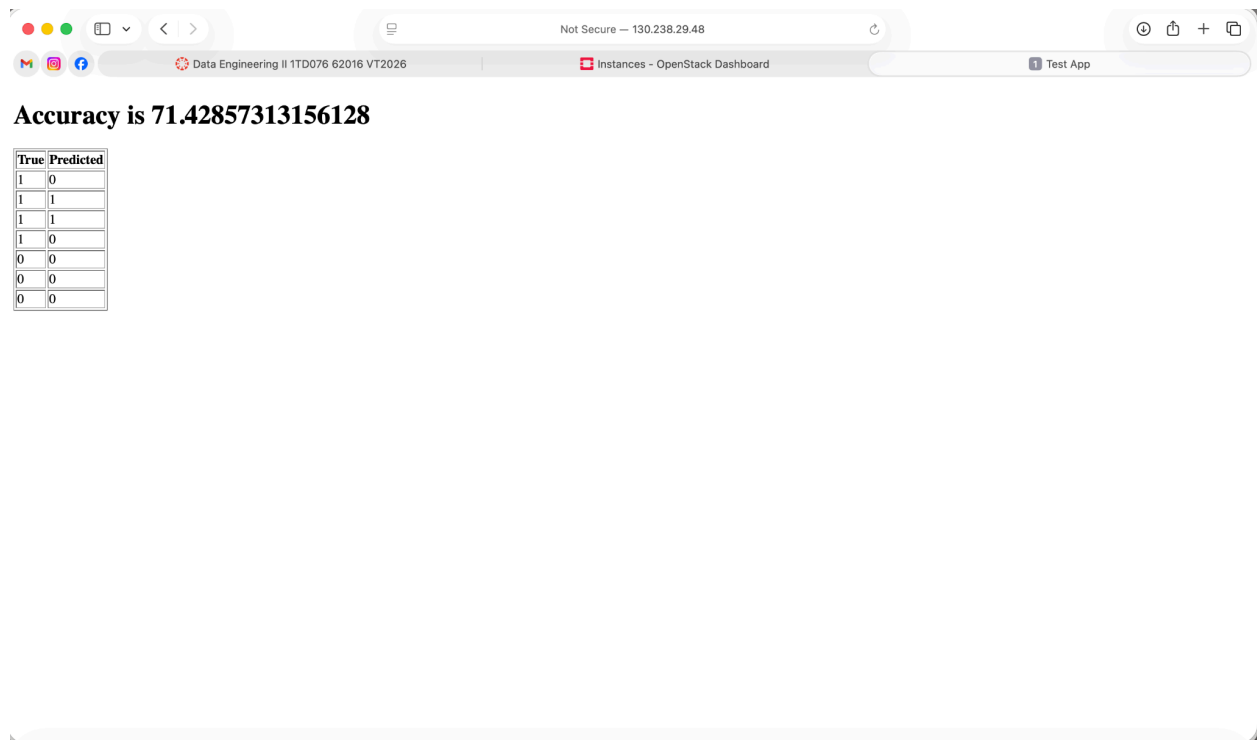
Question 3: The 10 to 15 minute boot time results from installing all dependencies at first boot. To address this, the most effective solution is to build a pre-baked custom VM image, also known as a "golden image," by installing all components on a base VM and then creating a reusable OpenStack image. With this approach, subsequent launches can skip the setup phase and be ready within seconds. Alternatively, using Docker, the application can be packaged into a

container image, pushed to a registry such as Docker Hub, and pulled by CloudInit to run directly. Additionally, a private pip mirror or package cache can further reduce download latency for repeated deployments. The most balanced approach, however, combines a pre-baked image with a minimal CloudInit script that handles only environment-specific configuration, such as secret injection.

Question 4 : This task involved deploying a machine learning model serving application on the Swedish Science Cloud. The deployment used a fully automated CloudInit contextualization strategy. Instead of manual configuration, the entire software stack, including RabbitMQ, Flask, Celery, TensorFlow, and the model serving repository, was provisioned at first boot using a cloud-cfg.txt file. Deployment was triggered from the client VM (client-vm-sotiris, floating IP 130.238.27.142) through the OpenStack Python SDK. The start_instance.py script handled authentication with Keystone and created the VM with the correct flavor, network, keypair, and user data, as shown in Figure 1. Once the instance transitioned from BUILD to ACTIVE, the production server was reachable at <http://130.238.29.48:5100>. The Flask /predictions endpoint was tested successfully via browser form submission. This returned predictions and a model accuracy of about 71.43%, as shown in Figure 2. After verification, the production VM was terminated as required. Screenshots document the workflow, including VM creation, SSH access, endpoint responses, and server listings. These are in the accompanying figures.



Welcome to the Machine Learning Course.



Task-2: Single server deployment with Docker Containers

Question 1: Task 2's Docker-based deployment resolves the main drawbacks of the raw CloudInit approach from Task 1. Deployment time is slashed. Instead of installing all packages and Python dependencies from scratch on each VM boot, Docker pulls prebuilt images from a registry and starts them instantly. This eliminates the 10–15-minute delay. Environment consistency is assured because the application and its dependencies are frozen in the container image. The same image runs identically across OS states, eliminating dependency drift. Service isolation improves, as each component—Flask, Celery, RabbitMQ—runs in its own container with a defined role and network interface. This surpasses putting all processes on the same OS environment. Finally, horizontal scaling becomes easy: adding Celery workers only requires a single `'docker compose up --scale worker_1=3 -d '` command, not extra VM setup and config.

Question 2: Despite the improvements, the Task 2 strategy is still limited in key ways. All containers run on one VM, so if the underlying hardware or operating system fails, the entire application stack stops working, and there is no automatic takeover by another machine (failover). There is no automated health monitoring or self-healing, meaning that if a container crashes, it must be restarted manually, unless you add extra configuration. Docker Compose does not support cross-host orchestration, meaning it cannot manage containers spread across multiple machines. This setup cannot scale beyond a single VM. There is no load balancing in front of the Flask container, so all requests go to a single web container regardless of the number of Celery workers running. Secrets management also remains basic, as RabbitMQ credentials are passed as plain environment variables, instead of using a secure secrets vault.

Question 3: The next step is to use a container orchestration platform such as Kubernetes or Docker Swarm. Kubernetes solves multi-host deployment by spreading containers across a cluster. It provides built-in health checks and automatic pod restarts. Load balancing is handled by Services and Ingress controllers. Secret management can be done with tools such as HashiCorp Vault or Kubernetes Secrets. For scaling Celery workers, a Kubernetes HorizontalPodAutoscaler can adjust worker replicas based on the depth of the RabbitMQ queue. Docker Swarm is a lighter alternative. It extends the Docker Compose model to a multi-node cluster with a few changes. Moving RabbitMQ to a managed message queue service reduces operational burden. This also eliminates the single broker as a point of failure.

Question 4: Vertical scalability means increasing the resources of a single machine — adding more CPU cores, RAM, or storage to handle a greater load. Horizontal scalability means adding

more instances of a service and distributing the workload across them. Task 2 follows horizontal scalability: rather than upgrading the VM, we spin up additional Celery worker containers using `docker compose up --scale worker_1=3 -d`. All three worker containers connect to the same RabbitMQ queue and independently consume tasks, so prediction requests are distributed across the pool. As shown in Figure 1, scaling to 3 workers creates `worker_1-1`, `worker_1-2`, and `worker_1-3` simultaneously, and scaling back down with `--scale worker_1=1` removes the additional instances without affecting the rest of the stack, as shown in the figure below.

```
ubuntu@prod-server-with-docker-6655:~$ cd /model_serving/single_server_with_docker/production_server
sudo docker compose up --scale worker_1=3 -d
WARN[0000] /model_serving/single_server_with_docker/production_server/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 5/5
  ✓ Container production_server-rabbit-1    Running      0.0s
  ✓ Container production_server-web-1       Running      0.0s
  ✓ Container production_server-worker_1-1  Running      0.0s
  ✓ Container production_server-worker_1-3  Started      2.3s
  ✓ Container production_server-worker_1-2  Started      2.8s
ubuntu@prod-server-with-docker-6655:/model_serving/single_server_with_docker/production_server$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
91fc53d06d24	production_server-worker_1	"celery -A workerA w..."	43 seconds ago	Up 41 seconds		producti
on_server-worker_1-3	production_server-worker_1	"celery -A workerA w..."	43 seconds ago	Up 41 seconds		producti
on_server-worker_1-2	production_server-worker_1	"celery -A workerA w..."	47 minutes ago	Up 47 minutes		producti
on_server-worker_1-1	production_server-web	"python ./app.py --h..."	47 minutes ago	Up 47 minutes	0.0.0.0:5100->5100/tcp, [::]:5100->5100/tcp	producti
4fd8c862d9a2	rabbitmq:3.11-management	"docker-entrypoint.s..."	47 minutes ago	Up 47 minutes	4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp	producti
on_server-rabbit-1						

```
ubuntu@prod-server-with-docker-6655:/model_serving/single_server_with_docker/production_server$
```

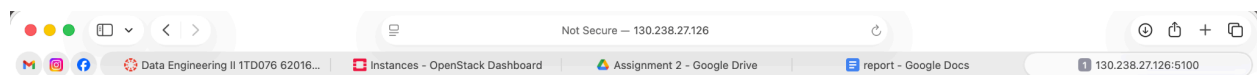
```
ubuntu@prod-server-with-docker-6655:/model_serving/single_server_with_docker/production_server$ sudo docker compose up --scale worker_1=1 -d
sudo docker ps
WARN[0000] /model_serving/single_server_with_docker/production_server/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 5/5
  ✓ Container production_server-rabbit-1    Running      0.0s
  ✓ Container production_server-worker_1-1  Running      0.0s
  ✓ Container production_server-worker_1-3  Removed     18.9s
  ✓ Container production_server-worker_1-2  Removed     18.9s
  ✓ Container production_server-web-1       Running      0.0s
```

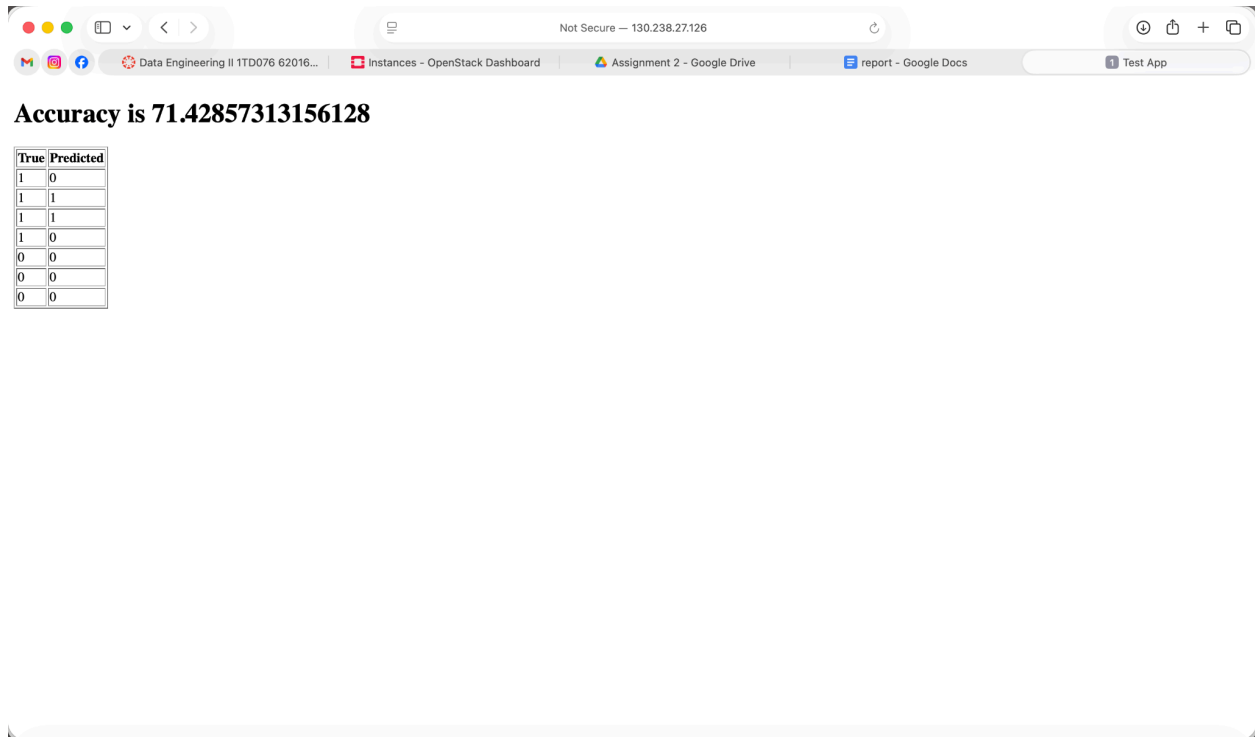
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
656de76809b5	production_server-worker_1	"celery -A workerA w..."	52 minutes ago	Up 52 minutes		producti
on_server-worker_1-1	production_server-web	"python ./app.py --h..."	52 minutes ago	Up 52 minutes	0.0.0.0:5100->5100/tcp, [::]:5100->5100/tcp	producti
4fd8c862d9a2	rabbitmq:3.11-management	"docker-entrypoint.s..."	52 minutes ago	Up 52 minutes	4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp	producti
on_server-rabbit-1						

```
ubuntu@prod-server-with-docker-6655:/model_serving/single_server_with_docker/production_server$
```

Question 5: This task repeated the model serving deployment from Task 1. It replaced the raw CloudInit provisioning strategy with a containerized approach using Docker and Docker Compose. The production server VM (prod_server_with_docker_6655, floating IP 130.238.27.126) was launched from the client VM using the same start_instance.py OpenStack script. A simplified cloud-cfg.txt was used that installs only Docker and clones the repository before executing docker-compose up -d. All application-level setup is handled by the Docker images themselves. This makes the CloudInit file significantly leaner than in Task 1.

Once the VM reached the ACTIVE state and CloudInit completed, three containers were running on the production server: production_server-rabbit-1, providing the RabbitMQ message broker on ports 5672 and 15672; production_server-web-1, serving the Flask frontend on port 5100; and production_server-worker_1-1, running the Celery worker that loads the Keras model and performs inference. The application was verified by accessing the welcome page and the /predictions endpoint, which returned an accuracy of 71.43%, consistent with Task 1, as shown in the figures below.





The horizontal scaling capability was then demonstrated by running `docker compose up --scale worker_1=3 -d`, which launched two additional worker containers (`worker_1-2` and `worker_1-3`) alongside the original, as confirmed by `docker ps`. All three workers connect to the same RabbitMQ queue and process tasks concurrently. The cluster was then scaled back down to a single worker using `--scale worker_1=1`, with the extra containers cleanly removed. The production VM was terminated upon task completion.

```
secon — ubuntu@prod-server-with-docker-6655: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.126 — 132x36
...nstack-client/single_node_with_docker_client — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 ...@prod-server-with-docker-6655: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.126 +

Memory usage: 31%          IPv4 address for ens3: 192.168.2.123
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@prod-server-with-docker-6655:~$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
656de76809b5   production_server-worker_1         "celery -A workerA w..." 27 minutes ago Up 27 minutes                                     producti
on_server-worker_1-1
5dd588348ead   production_server-web              "python ./app.py --h..." 27 minutes ago Up 27 minutes   0.0.0.0:5100->5100/tcp, [::]:5100/tcp  producti
on_server-web-1
4fd8c862d9a2   rabbitmq:3.11-management          "docker-entrypoint.s..." 27 minutes ago Up 27 minutes   4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp  producti
on_server-rabbit-1
ubuntu@prod-server-with-docker-6655:~$
```

Task-3: Deployment of multiple servers using Ansible

Question 1: CloudInit and Ansible serve fundamentally different roles in the provisioning lifecycle. CloudInit is a **boot-time contextualization tool** that runs automatically the first time a VM starts, before any remote connection to the machine is possible. It is tightly coupled to the cloud platform's instance metadata service and handles early initialization tasks such as creating user accounts, injecting SSH keys, setting hostnames, and installing base packages from a YAML cloud-config file. It cannot be rerun without reimaging the instance, and it operates in isolation on a single VM, unaware of other machines in the environment.

Ansible, by contrast, is a **post-boot configuration management and orchestration tool** that runs from a dedicated controller node over SSH once the target VMs are reachable. It is agentless — no software is required on the managed hosts beyond Python, and an SSH daemon — and its playbooks are **idempotent**, meaning they can be re-executed safely without producing unintended side effects. Critically, Ansible can target **multiple hosts simultaneously**, applying different configurations to different groups within a single playbook run. In Task 3, this distinction is made explicit: CloudInit does only the minimum required at boot (creating appuser

and injecting the cluster SSH key), while Ansible performs all application-level configuration — installing Docker on the production server, running `docker compose up -d`, and installing ML packages on the development server — against both VMs in parallel from the client machine.

Question 2: Both `prod-cloud-cfg.txt` and `dev-cloud-cfg.txt` are intentionally minimal cloud-config files. They are structurally identical and serve a single purpose: bootstrapping the conditions that allow Ansible to connect and take over. Each file begins with the `#cloud-config` directive, which identifies it as a CloudInit configuration to the instance metadata service. The `users` block creates a non-root operating system user named `appuser`, grants it passwordless `sudo` privileges (`sudo: ALL=(ALL) NOPASSWD:ALL`), assigns `/bin/bash` as its shell, and injects the cluster's SSH public key into its `~/.ssh/authorized_keys` via the `ssh_authorized_keys` field. The `byobu_default: system` directive enables the Byobu terminal multiplexer as the default shell wrapper at the system level. Notably, neither file contains any `packages` block or `runcmd` directives — there is no package installation, no repository cloning, and no service startup at boot. All of that responsibility is delegated to the Ansible playbook (`configuration.yml`), which runs after both VMs reach the `ACTIVE` state. The separation is deliberate: CloudInit handles identity and access, while Ansible handles the environment and applications.

Question 3: The core problem Ansible solves in this task is **multi-host configuration management from a single control point**. In Tasks 1 and 2, each production server required its own CloudInit script to install and start the entire application stack independently at boot. This approach does not scale: if the deployment involves two, five, or fifty servers with different roles, maintaining separate CloudInit scripts per VM becomes error-prone, and any configuration change requires re-imaging every affected instance. With Ansible, the client VM becomes a centralized orchestrator: a single `ansible-playbook` command applies role-specific configurations to both the production and development servers simultaneously, using host groups (`prodserver`, `devserver`) in the inventory to differentiate between them. The production server receives Docker, the model-serving Docker Compose stack, and an updated `/etc/hosts` file; the development server receives the Python ML dependencies. Should a configuration need to change, only the playbook needs updating, and it can be re-run idempotently across all nodes without reprovisioning. This also eliminates the 10–15 minute CloudInit delay for application-level setup, since CloudInit now only performs the lightweight user and key injection, and Ansible handles the rest in parallel.

```

ubuntu@client-vm-sotiris:~$ whoami # should print: ubuntu

mkdir -p /home/ubuntu/cluster-keys
ssh-keygen -t rsa -f /home/ubuntu/cluster-keys/cluster-key -N ""
ubuntu
Generating public/private rsa key pair.
Your identification has been saved in /home/ubuntu/cluster-keys/cluster-key
Your public key has been saved in /home/ubuntu/cluster-keys/cluster-key.pub
The key fingerprint is:
SHA256:QuC07tvJAZ5gINBcPmYZz9a2i/PHh0ajo6MerTd43/c ubuntu@client-vm-sotiris
The key's randomart image is:
+---[RSA 3072]-----+
|.o . = |
|. o+ B . |
|o 0 = o |
|.. + + . . |
| o o . S. |
| . = o oo.. |
| .. = +o.o |
| .O=+.*o + |
| .++o=*oo+ .E |
+---[SHA256]-----+
ubuntu@client-vm-sotiris:~$ cat /home/ubuntu/cluster-keys/cluster-key.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDtXL6mybbxUsl5kwILPxUrUwIXWraKKoK0kSTGRzECV2XIh0jNr95UwDFru5AV6rKWLvMQPsvAp/W1v+mv6DIhQ/aOE
J0yiUe1y9QkF4dWgOM1gt8eLVl90kCjeMwGrzf4KX5nTpukTsF9aLwUL1AIkROHG633M/LPRrN5cu2AhIdfi1ynIT0nxIY4g6nfiJ9kgcpE8gdcaSf9gWRWkXo4/MAB5FYu
8DRYYgFWstDIV989w1Hb3rWBx8cV8CnU1+FeS/SQhoodHiN/Jn0u/HrfbBZJTHff7q4aUHvQ3v6uHORHzsKM8KvAeX55domfNpB3UKLFEcux/N2n7ytKptYsqD8PPhupM0i
OYgJfJj4ELaTsF/lw6S78FNk6P/kE3p3CFz4dTRaGxNIGxu/W6VCFdyBsSyL4z9wmswRUdoMyRYB8RX9yCWLYA2tgcElSbxPoumZYDxZlHg+mcd4RkISULF0rSYz78C7eMgw
4HUgw0WwJCDc6jLLQDlILHc= ubuntu@client-vm-sotiris
ubuntu@client-vm-sotiris:~$

```

```

ubuntu@client-vm-sotiris:~/model_serving/openstack-client/single_node_with_docker_ansiible_client$ python3 start_instances.py
user authorization completed.
Creating instances ...
waiting for 10 seconds..
Instance: prod_server_with_docker_2111 is in BUILD state, sleeping for 5 seconds more...
Instance: dev_server_2111 is in BUILD state, sleeping for 5 seconds more...
Instance: prod_server_with_docker_2111 is in ACTIVE state ip address: 192.168.2.7
Instance: dev_server_2111 is in ACTIVE state ip address: 192.168.2.245
ubuntu@client-vm-sotiris:~/model_serving/openstack-client/single_node_with_docker_ansiible_client$

```

```
secon — ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 — 143x47
~ — ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 ...ver-with-docker-6655: /model_serving/single_server_with_docker/production_server — ~zsh

ubuntu@client-vm-sotiris:~$ ssh -i /home/ubuntu/cluster-keys/cluster-key appuser@192.168.2.7 using Ansible
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-91-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Tue May  5 13:40:40 UTC 2026
System load:  0.0185546875   Processes: 100
Usage of /:   7.5% of 19.20GB Users logged in: 0
Memory usage: 5%           IPv4 address for ens3: 192.168.2.7
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.
If you want to know more about Ansible, visit following URLs:
0 updates can be applied immediately. - https://www.ansible.com/blog/it-automation
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status ble Works
- How to Install and Configure Ansible on Ubuntu 18.04 | DigitalOcean

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

1. Login to the Client machine from your local machine

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Tue May  5 13:39:04 2026 from 192.168.2.158
appuser@prod-server-with-docker-2111:~$
```

```
second — ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 — 155x50
-- ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 ...lth-docker-6655: /model_serving/single_server_with_docker/production_server -- -zsh

ubuntu@client-vm-sotiris:~$ ssh -i /home/ubuntu/cluster-keys/cluster-key appuser@192.168.2.245
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-91-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Tue May  5 13:41:56 UTC 2026

System load:  0.0               Processes:    98
Usage of /:   7.5% of 19.20GB   Users logged in: 0
Memory usage: 5%              IPv4 address for ens3: 192.168.2.245
Swap usage:  0%

user authorization completed.

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.
  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

1. Login to the Client machine from your local machine

2. Go to model_serving/ci_cd. This directory contains the following two sub-directories:

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

ask Application based frontend

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

-- app.py
-- static
-- templates

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

run task.py

-- workerA.py

ask Model and Data

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

-- model.h5
-- model.json

appuser@dev-server-2111:~$
```

```
second — ubuntu@client-vm-sotiris: ~/model_serving/openstack-client/single_node_with_docker_ansi...
.../single_node_with_docker_ansi_client — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 ...-server-with-docker-6655: /model_serving/single_server_with_docker/production_server — ~zsh

changed: [prodserver]

TASK [Add Docker repository] *****
changed: [prodserver]

TASK [apt update] *****
ok: [prodserver]

TASK [Create Ansible docker config directory.] *****
changed: [prodserver]

TASK [Create Ansible docker file to configure Docker MTU to match SNIC for pip issue] *****
changed: [prodserver]

TASK [Install Docker] *****
changed: [prodserver]

TASK [Running containers] *****
changed: [prodserver]

PLAY [devserver] *****

TASK [Gathering Facts] *****
ok: [devserver]

TASK [Extra packages] *****
changed: [devserver]

TASK [Install ML packages] *****
changed: [devserver]

PLAY RECAP *****
devserver      : ok=8   changed=6   unreachable=0   failed=0   skipped=0   rescued=0   ignored=0
prodserver     : ok=15  changed=11  unreachable=0   failed=0   skipped=0   rescued=0   ignored=0

ubuntu@client-vm-sotiris:~/model_serving/openstack-client/single_node_with_docker_ansi_client$
```

east-1.cloud.snic.se

Data Engineering II 1TD076 62016... Instances - OpenStack Dashboard Assignment 2 - Google Drive report - Google Docs Grammarly

UPPMAX 2026/1-24

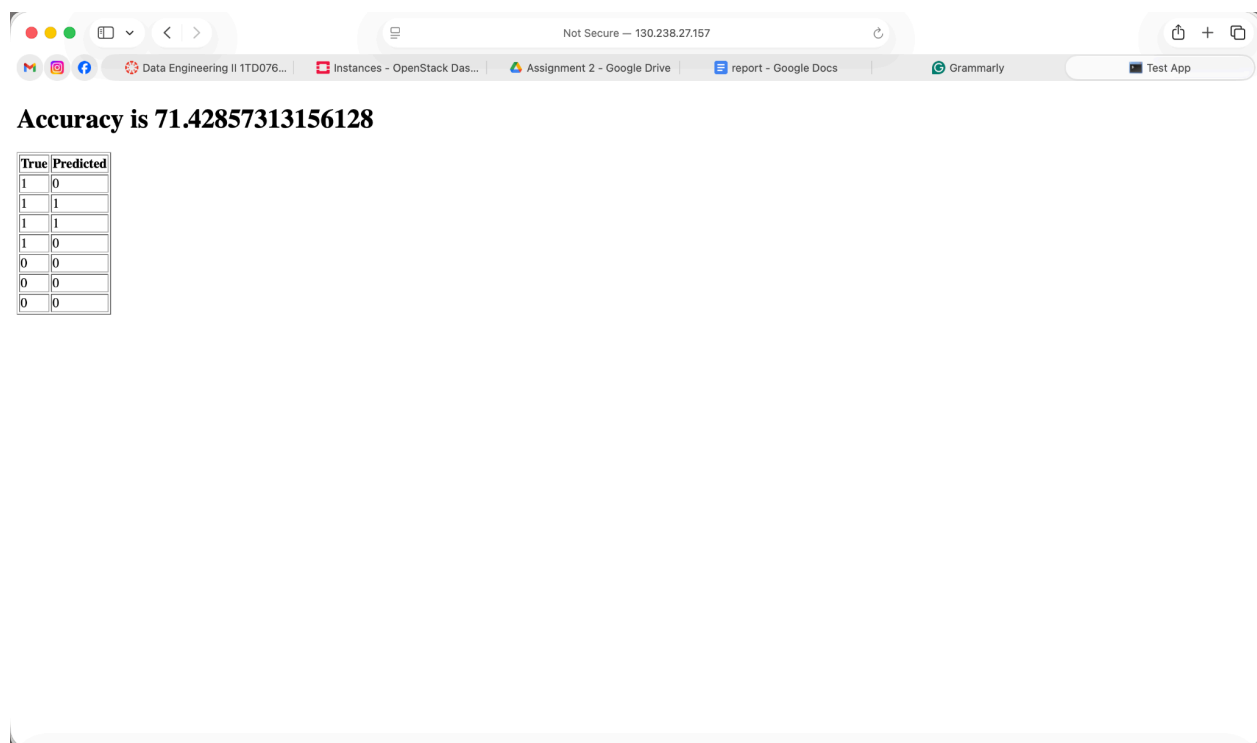
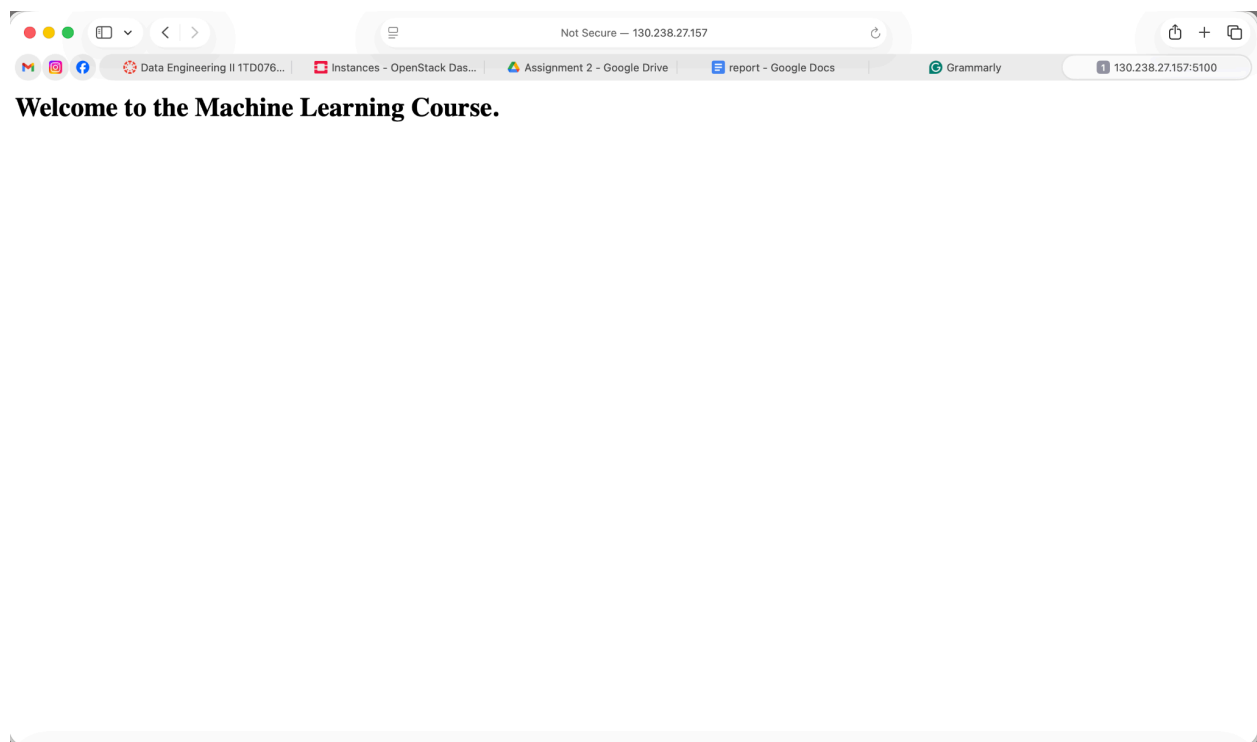
Project / Compute / Instances

Instances

IPv4 Address = 192.168.2.7 Filter Launch Instance Delete Instances More Actions

Displaying 9 items

	Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
☐	dev_server_7101	Ubuntu 22.04 - 2024.0 1.15	192.168.2.77	ssc.medium	de1-course-key	Active	az1 nova	None	Running	47 minutes	Create Snapshot
☐	prod_server_with_docker_2111	Ubuntu 22.04 - 2024.0 1.15	192.168.2.7, 130.238.27.157	ssc.medium	de2-course-key2	Active	az1 nova	None	Running	1 hour, 11 minutes	Create Snapshot



Task-4: CI/CD using Git HOOKS

Question 1: Git hooks are scripts stored in a repository's `hooks/` directory. Git runs these scripts automatically at important steps: before a commit, before a push, or after a push to a remote server. You don't need extra tools; Git handles hooks itself. If a client-side hook ends with an error code, Git stops the operation. Server-side hooks like `post-receive` can't stop the push after it's received. Instead, they are used to start automated deployment. In this task, the `post-receive` hook reads each updated reference from standard input. Each line looks like `<old-sha> <new-sha> <refname>`. The script checks each line to see whether the reference ends with `'master'`. If it does, it prints a confirmation and runs `sudo git --work-tree=/model_serving/ci_cd/production_server --git-dir=/home/appuser/my_project checkout -f`. This command updates the live application directly, with no additional file transfer required. Pushes to other branches are noted but ignored, so only master triggers deployment. The figure below shows the hook script set up on the production server, with correct content and executable permissions.

```
secon — ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 — 132x37
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/appuser/my_project/
appuser@prod-server-with-docker-2111:~/my_project$ cat > /home/appuser/my_project/hooks/post-receive << 'EOF'
#!/bin/bash
while read oldrev newrev ref
do
    if [[ $ref =~ .*/master$ ]];
    then
        echo "Master ref received. Deploying master branch to production..."
        sudo git --work-tree=/model_serving/ci_cd/production_server --git-dir=/home/appuser/my_project checkout -f
    else
        echo "Ref $ref successfully received. Doing nothing: only the master branch may be deployed on this server."
    fi
done
EOF
appuser@prod-server-with-docker-2111:~/my_project$ chmod +x /home/appuser/my_project/hooks/post-receive
appuser@prod-server-with-docker-2111:~/my_project$ cat /home/appuser/my_project/hooks/post-receive
#!/bin/bash
while read oldrev newrev ref
do
    if [[ $ref =~ .*/master$ ]];
    then
        echo "Master ref received. Deploying master branch to production..."
        sudo git --work-tree=/model_serving/ci_cd/production_server --git-dir=/home/appuser/my_project checkout -f
    else
        echo "Ref $ref successfully received. Doing nothing: only the master branch may be deployed on this server."
    fi
done
appuser@prod-server-with-docker-2111:~/my_project$
```

Question 2: The bare repository at `/home/appuser/my_project` on the production server (192.168.2.7) serves as the Git remote endpoint. This is the target the development server connects to when running `git push production master`. A bare repository, initialized with `git init --bare`, stores only the Git object database (commits, blobs, and trees) and has no associated working tree. This is the correct structure for a shared or server-side remote because it can safely receive pushes from multiple clients. There is no risk of working-tree conflicts. Critically, server-side hooks such as post-receive are tied to this repository. These hooks live in the `hooks/` directory and trigger when the repository receives a push. Without this bare repository, there would be no remote for the development server to push to. The deployment hook either did not exist or could not be triggered. The separation of concerns is intentional. The bare repo handles the Git transfer and hook execution. The actual application files are checked out into a separate directory (`/model_serving/ci_cd/production_server/`) via the `--work-tree` flag. This keeps the two concerns entirely decoupled. As shown in the figure below, the bare repository was successfully initialized on the production server.

```
secon — ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 — 132x37

System information as of Tue May  5 16:31:35 UTC 2026

System load:  0.03          Processes:           126
Usage of /:   33.7% of 19.20GB Users logged in:       1
Memory usage: 33%          IPv4 address for ens3: 192.168.2.7
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

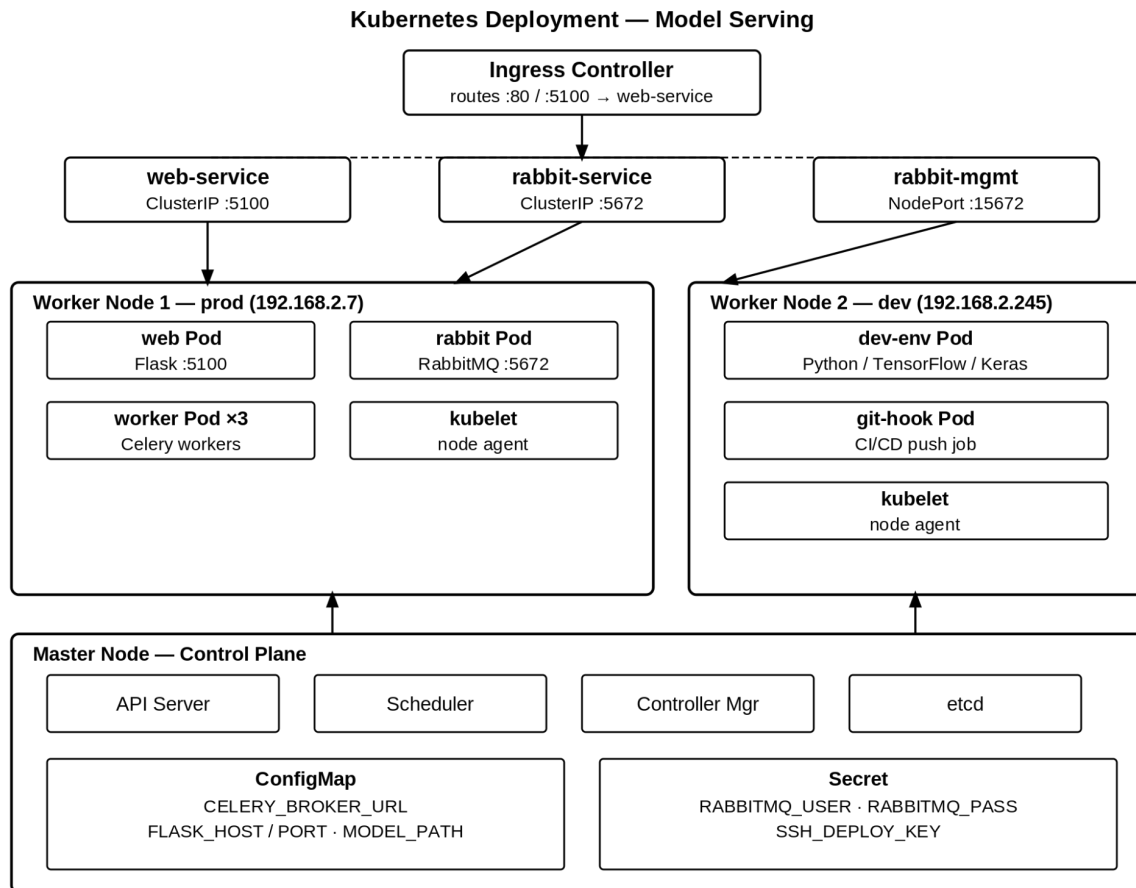
*** System restart required ***
Last login: Tue May  5 16:28:47 2026 from 192.168.2.245
appuser@prod-server-with-docker-2111:~$ mkdir /home/appuser/my_project
cd /home/appuser/my_project
git init --bare
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/appuser/my_project/
appuser@prod-server-with-docker-2111:~/my_project$
```

Question 3: pre-commit is a client-side hook that executes immediately before a commit object is created. It receives no arguments and is typically used to enforce code quality standards — running linters, static analyzers, or test suites — and aborts the commit with a non-zero exit if any check fails. commit-msg runs after the developer writes a commit message but before the commit is finalized. It receives the path to the temporary file containing the proposed message and is used to validate its format, for example, enforcing the presence of a ticket identifier or a maximum subject line length. pre-push fires on the client side just before objects are transmitted to the remote server. It can be used to run a full test suite or to block pushes to protected branches; a non-zero exit prevents any data from being sent. post-receive is a server-side hook that runs after a push has been fully received and all references updated on the remote. It cannot abort the push, but it is the standard trigger for deployment automation. In this task, it checked whether the pushed reference was the master branch and, if so, checked out the committed working tree into the live production directory — as confirmed in the figure below.

```
[master 432d2e7] updated model - changed epochs
2 files changed, 1 insertion(+), 1 deletion(-)
rewrite model.json (100%)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.11 KiB | 719.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Master ref received. Deploying master branch to production...
To 192.168.2.7:/home/appuser/my_project
e314fb6..432d2e7 master -> master
appuser@dev-server-2111:~/my_project$
```

Question 4: Mapping the Task 3 multi-server setup to Kubernetes requires translating each functional component into a corresponding Kubernetes object. The three Docker containers running on the production server — the Flask web application, the Celery worker, and the RabbitMQ broker — become separate Deployment resources, each managing a set of replicated pods. The Flask web Deployment is exposed to external traffic via an Ingress controller, replacing the manual floating IP attachment used in Task 3. Internal communication between components (e.g., Celery workers connecting to RabbitMQ) is handled by ClusterIP Services, while the RabbitMQ management interface is exposed via a NodePort Service for monitoring. The development server's Python/TensorFlow training environment and the CI/CD push job each become their own Deployments, scheduled on dedicated worker nodes. Non-sensitive runtime configuration — broker URLs, model paths, Flask port — is stored in a ConfigMap and injected as environment variables, keeping container images generic and environment-agnostic. Sensitive values, specifically the RabbitMQ username and password and the SSH deploy key used by the

CI/CD pipeline, are stored in a Secret, base64-encoded, and mounted only into the pods that require them. The control plane (API server, scheduler, controller manager, etcd) manages desired state continuously across all worker nodes, providing automatic pod restarts, declarative Celery worker scaling, and rolling model updates — capabilities absent from the Docker Compose setup in Task 3.



Question 5: Task 4 implemented a Git-based CI/CD pipeline that automates the delivery of updated machine learning models from the development server to the production environment, eliminating all manual file transfer and service intervention. The first step was to establish passwordless SSH communication between the development and production servers. An RSA 3072-bit keypair was generated on the development server using `ssh-keygen`, saving the private key to the default path `/home/appuser/.ssh/id_rsa`. The resulting public key was appended to `/home/appuser/.ssh/authorized_keys` on the production server, after which direct SSH from dev to prod required no password, as confirmed in the next figure.

```
secon — ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 — 149x56

ubuntu@client-vm-sotiris:~$ ssh -i /home/ubuntu/cluster-keys/cluster-key appuser@192.168.2.245
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-91-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue May  5 16:28:40 UTC 2026

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge
   To Automate Development and Deployment Tasks | DigitalOcean

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required *** We have created an empty git repository on the production server. Why we need that
Last login: Tue May  5 16:27:11 2026 from 192.168.2.158
appuser@dev-server-2111:~$ ssh appuser@192.168.2.7
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-91-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue May  5 16:28:46 UTC 2026

System load:  0.02          Processes:           126
Usage of /:   33.7% of 19.20GB   Users logged in:    1
Memory usage: 32%            IPv4 address for ens3: 192.168.2.7
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Tue May  5 16:27:17 2026 from 192.168.2.245
appuser@prod-server-with-docker-2111:~$
```

On the production server, a bare Git repository was initialized at /home/appuser/my_project using git init --bare. The post-receive hook was written to its hooks/ directory and made executable. The hook checks whether the pushed branch is master; if so, it uses git checkout -f with --work-tree pointing to /model_serving/ci_cd/production_server/ to deploy the committed files directly into the running application directory. On the development server, a local repository was initialized at /home/appuser/my_project, the initial model.h5 and model.json files were committed, and the production bare repository was registered as a remote named production. The first push to master transferred the objects and immediately triggered the hook, printing remote: Master ref received. Deploying master branch to production..., confirming that the pipeline was operational from the outset, as shown in the figure below.

secon — ubuntu@client-vm-sotiris: ~ — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 — 132x37

```
git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/appuser/my_project/.git/
appuser@dev-server-2111:~/my_project$ cp /model_serving/ci_cd/development_server/model.h5 /home/appuser/my_project/
cp /model_serving/ci_cd/development_server/model.json /home/appuser/my_project/
appuser@dev-server-2111:~/my_project$ git config user.email "s35457@student.uu.se"
git config user.name "appuser"
appuser@dev-server-2111:~/my_project$ git config user.email "s35457@student.uu.se"
appuser@dev-server-2111:~/my_project$ git config user.name "appuser"
appuser@dev-server-2111:~/my_project$ cd /home/appuser/my_project
git add .
git commit -m "initial model"
[master (root-commit) e314fb6] initial model
 2 files changed, 1 insertion(+)
 create mode 100644 model.h5
 create mode 100644 model.json
appuser@dev-server-2111:~/my_project$ git remote add production appuser@192.168.2.7:/home/appuser/my_project
git push production master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.36 KiB | 2.36 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
remote: Master ref received. Deploying master branch to production...
To 192.168.2.7:/home/appuser/my_project
 * [new branch]      master -> master
appuser@dev-server-2111:~/my_project$
```

To validate the end-to-end pipeline, `neural_net.py` was modified to increase the number of training epochs from 150 to 200, and the model was retrained on the development server. The updated model achieved a training accuracy of 78.56% at epoch 200/200.

```
secon — ubuntu@client-vm-sotiris: ~/model_serving/ci_cd/development_server — ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.142 — 132x37
Epoch 188/200
77/77 [=====] - 0s 1ms/step - loss: 0.4713 - accuracy: 0.7725
Epoch 189/200
77/77 [=====] - 0s 1ms/step - loss: 0.4702 - accuracy: 0.7765
Epoch 190/200
77/77 [=====] - 0s 1ms/step - loss: 0.4682 - accuracy: 0.7817
Epoch 191/200
77/77 [=====] - 0s 1ms/step - loss: 0.4687 - accuracy: 0.7778
Epoch 192/200
77/77 [=====] - 0s 1ms/step - loss: 0.4656 - accuracy: 0.7843
Epoch 193/200
77/77 [=====] - 0s 1ms/step - loss: 0.4675 - accuracy: 0.7765
Epoch 194/200
77/77 [=====] - 0s 1ms/step - loss: 0.4669 - accuracy: 0.7712
Epoch 195/200
77/77 [=====] - 0s 1ms/step - loss: 0.4581 - accuracy: 0.7895
Epoch 196/200
77/77 [=====] - 0s 1ms/step - loss: 0.4735 - accuracy: 0.7752
Epoch 197/200
77/77 [=====] - 0s 1ms/step - loss: 0.4660 - accuracy: 0.7778
Epoch 198/200
77/77 [=====] - 0s 1ms/step - loss: 0.4593 - accuracy: 0.7817
Epoch 199/200
77/77 [=====] - 0s 1ms/step - loss: 0.4672 - accuracy: 0.7608
Epoch 200/200
77/77 [=====] - 0s 1ms/step - loss: 0.4656 - accuracy: 0.7830
24/24 [=====] - 0s 1ms/step - loss: 0.4554 - accuracy: 0.7856
Accuracy: 78.56
Saved model to disk
Loaded model from disk
24/24 [=====] - 0s 1ms/step
[6.0, 148.0, 72.0, 35.0, 0.0, 33.6, 0.627, 50.0] => 1 (expected 1)
[1.0, 85.0, 66.0, 29.0, 0.0, 26.6, 0.351, 31.0] => 0 (expected 0)
[8.0, 183.0, 64.0, 0.0, 0.0, 23.3, 0.672, 32.0] => 1 (expected 1)
[1.0, 89.0, 66.0, 23.0, 94.0, 28.1, 0.167, 21.0] => 0 (expected 0)
[0.0, 137.0, 40.0, 35.0, 168.0, 43.1, 2.288, 33.0] => 1 (expected 1)
appuser@dev-server-2111:/model_serving/ci_cd/development_server$
```

The new model.h5 and model.json files were committed and pushed; the output again confirmed hook execution with the line "remote: Master ref received...". The production predictions page at <http://130.238.27.157:5100/predictions> reflected the model update immediately: accuracy improved from 71.43% before the push to 85.71% after, with a visible change in the prediction output for the last sample, as shown in the figures below.

The exercise demonstrated that a Git hook pipeline provides a reproducible, auditable deployment path where every push to master constitutes a documented and automatically applied production update.